



CS471 – Web Technologies

Lab 1: Network Traffic Analysis Using Wireshark

Student Name: Amsha Aali Alrashidi

Student ID: 431201367

Section: 5529

Part 1: Capturing HTTP Traffic.

Task 1: Start Wireshark and capture packets.

Task 2: Filter HTTP packets and analyze them

The image displays two screenshots of the Wireshark network protocol analyzer. The top screenshot shows a list of captured packets in the packet list pane, with packet 206 selected. The packet details pane shows the structure of the selected packet, including Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data of the packet. The bottom screenshot shows the same packet list, but with packet 214 selected. The packet details pane shows the structure of the selected packet, including Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data of the packet. The bottom status bar indicates that 297 packets are displayed, with 6 (2.0%) displayed and 0 (0.0%) dropped.

Wireshark interface showing captured HTTP traffic. The packet list pane displays several packets, with packet 206 selected. The packet details pane shows the structure of the selected packet, including Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data of the packet.

Wireshark interface showing captured HTTP traffic. The packet list pane displays several packets, with packet 214 selected. The packet details pane shows the structure of the selected packet, including Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol. The packet bytes pane shows the raw data of the packet.

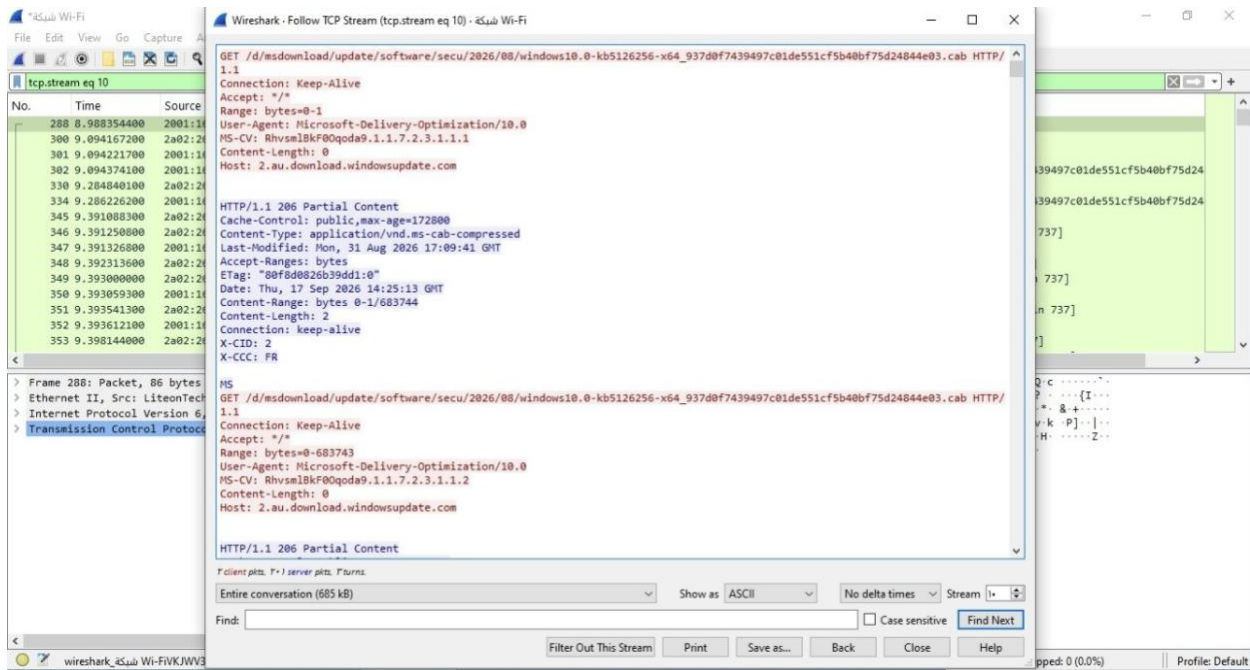
An HTTP packet was selected and analyzed.

The following information was observed:

- **HTTP Method:** GET
- **Request URI:** /online
- **HTTP Version:** HTTP/1.1
- **Response Status:** 200 OK

Part 2: Analyzing TCP/IP Traffic.

Task 1: Filter TCP packets

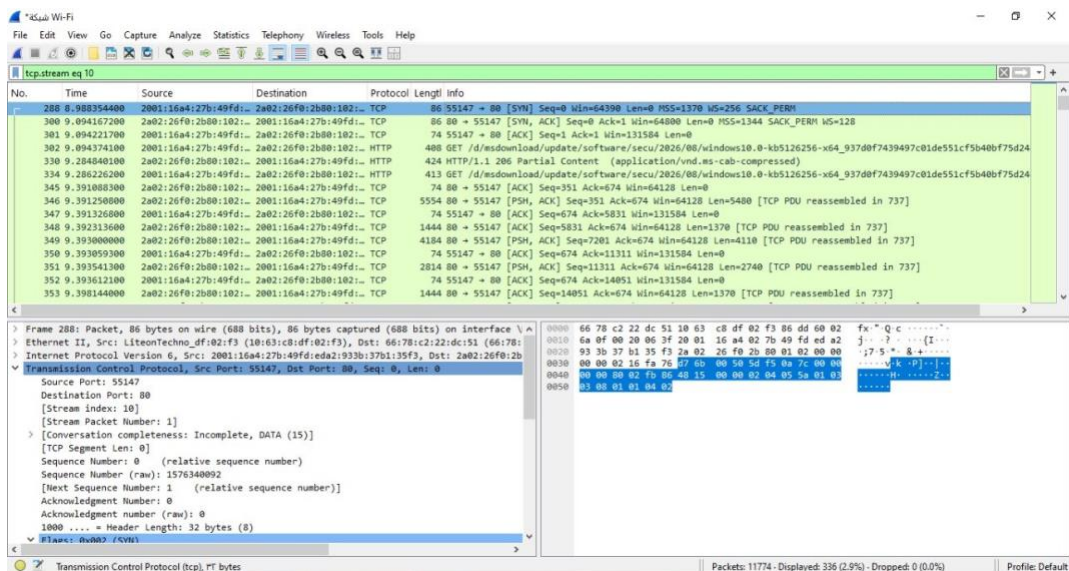


Task 2: Analyze TCP handshake and investigate Data Transfer and Termination.

Step 1: Find and select packets related to the TCP three-way handshake:

Step 2: Note the sequence and acknowledgment numbers.

SYN: Initiates a connection. SYN: Seq = 0, ack = 0



SYN-ACK: Acknowledges and responds to the SYN. SYN-ACK: Seq = 0, Ack = 1

Wireshark packet capture showing a SYN-ACK packet (Frame 300) from 2001:16a4:27b:49fd to 2a02:26f0:2b80:102. The packet is a TCP SYN-ACK with Seq=0 and Ack=1. The packet details pane shows the TCP header and the payload (86 bytes). The packet bytes pane shows the raw data.

No.	Time	Source	Destination	Protocol	Length	Info
288	0.988354400	2001:16a4:27b:49fd	2a02:26f0:2b80:102	TCP	86	55147 → 80 [SYN] Seq=0 Win=64390 Len=0 MSS=1370 WS=256 SACK_PERM
300	0.994167200	2a02:26f0:2b80:102	2001:16a4:27b:49fd	TCP	86	80 → 55147 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1344 SACK_PERM WS=128

ACK: Acknowledges the SYN-ACK and establishes the connection. ACK: Seq = 1, Ack = 1

Wireshark packet capture showing an ACK packet (Frame 301) from 2001:16a4:27b:49fd to 2a02:26f0:2b80:102. The packet is a TCP ACK with Seq=1 and Ack=1. The packet details pane shows the TCP header and the payload (74 bytes). The packet bytes pane shows the raw data.

No.	Time	Source	Destination	Protocol	Length	Info
288	0.988354400	2001:16a4:27b:49fd	2a02:26f0:2b80:102	TCP	86	55147 → 80 [SYN] Seq=0 Win=64390 Len=0 MSS=1370 WS=256 SACK_PERM
300	0.994167200	2a02:26f0:2b80:102	2001:16a4:27b:49fd	TCP	86	80 → 55147 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1344 SACK_PERM WS=128
301	0.994221700	2001:16a4:27b:49fd	2a02:26f0:2b80:102	TCP	74	55147 → 80 [ACK] Seq=1 Ack=1 Win=131584 Len=0

Step 3: Observe the data packets exchanged between the client and server.

The data packets were marked with PSH, ACK flags, indicating actual data being pushed between client and server.

tcp.stream eq 10

No.	Time	Source	Destination	Protocol	Length	Info
288	8.988354400	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	TCP	86	55147 → 80 [SYN] Seq=0 Win=64390 Len=0 MSS=1370 WS=256 SACK_PERM
300	9.094167200	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	86	80 → 55147 [SYN, ACK] Seq=0 Ack=1 Win=64800 Len=0 MSS=1344 SACK_PERM WS=128
301	9.094221700	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	TCP	74	55147 → 80 [ACK] Seq=1 Ack=1 Win=131584 Len=0
302	9.094374100	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	HTTP	408	GET /d/MSdownload/update/software/secu/2026/08/windows10.0-kb5126256-x64_937d0f7439497c01de551c5b40bf75d24
330	9.284840100	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	HTTP	424	HTTP/1.1 206 Partial Content (application/vnd.ms-cab-compressed)
334	9.286226200	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	HTTP	413	GET /d/MSdownload/update/software/secu/2026/08/windows10.0-kb5126256-x64_937d0f7439497c01de551c5b40bf75d24
345	9.391088300	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	74	80 → 55147 [ACK] Seq=351 Ack=674 Win=64128 Len=0
346	9.391250000	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	5554	80 → 55147 [PSH, ACK] Seq=351 Ack=674 Win=64128 Len=5480 [TCP PDU reassembled in 737]
347	9.391326000	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	TCP	74	55147 → 80 [ACK] Seq=674 Ack=5831 Win=131584 Len=0
348	9.392313600	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	1444	80 → 55147 [ACK] Seq=5831 Ack=674 Win=64128 Len=1370 [TCP PDU reassembled in 737]
349	9.393000000	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	4184	80 → 55147 [PSH, ACK] Seq=7201 Ack=674 Win=64128 Len=4110 [TCP PDU reassembled in 737]
350	9.393059300	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	TCP	74	55147 → 80 [ACK] Seq=674 Ack=11311 Win=131584 Len=0
351	9.393541300	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	2814	80 → 55147 [PSH, ACK] Seq=11311 Ack=674 Win=64128 Len=2740 [TCP PDU reassembled in 737]
352	9.393612100	2001:16a4:27b:49fd::	2a02:26f0:2b80:102::	TCP	74	55147 → 80 [ACK] Seq=674 Ack=14051 Win=131584 Len=0
353	9.398144000	2a02:26f0:2b80:102::	2001:16a4:27b:49fd::	TCP	1444	80 → 55147 [ACK] Seq=14051 Ack=674 Win=64128 Len=1370 [TCP PDU reassembled in 737]

Acknowledgment Number: 674 (relative ack number)
Acknowledgment number (raw): 1576340766
0101 = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
.....0000..... = Reserved: Not set
.....0..... = Accurate ECN: Not set
.....0..... = Congestion Window Reduced: Not set
.....0..... = ECN-Echo: Not set
.....0..... = Urgent: Not set
.....1..... = Acknowledgment: Set
.....1..... = Push: Set
.....0..... = Reset: Not set
.....0..... = SYN: Not set
.....0..... = FIN: Not set
[TCP Flags:AP....]
Window: 581
[Calculated window size: 64128]

0040 0d 1e 50 10 01 f5 08 be 00 00 48 54 54 50 2f 31HTTP/1
0050 2e 31 20 32 30 36 20 50 61 72 74 69 61 6c 20 431 206 P artial C
0060 6f 6e 74 65 6e 74 0d 0a 43 61 63 68 65 2d 43 6ftent:~ Cache-C
0070 6e 74 72 6f 6c 3a 20 70 75 62 6c 69 63 2c 6d 61ntrol: p ublic,ma
0080 78 2d 61 67 65 3d 31 37 32 38 30 30 0d 0a 43 6fx-age=17 2800~ Co
0090 6e 74 65 6e 74 2d 54 79 70 65 3a 20 61 70 70 6cntent~ty pes: appl
00a0 69 63 61 74 69 6f 6e 2f 76 6e 64 2e 6d 73 2d 63ication/ vnd.ms-c
00b0 61 62 2d 63 6f 6d 70 72 65 73 73 65 64 0d 0a 4cab-compr essed~L
00c0 61 73 74 2d 4d 6f 64 69 66 69 65 64 3a 20 4d 6fast-Modi fied: Mo
00d0 6e 2c 20 33 31 20 41 75 67 20 32 30 32 36 20 31n, 31 Au g 2026~1
00e0 37 3a 30 39 3a 34 31 20 47 4d 54 0d 0a 41 63 637:09:41 GMT~Acc
00f0 65 70 74 2d 52 61 6e 67 65 73 3a 20 62 79 74 65ept-Rang es: byte
0100 73 0d 0a 45 54 61 67 3a 20 22 38 30 66 38 64 30s~Etag: "80f6d0
0110 38 32 36 62 33 39 64 64 31 3a 30 22 0d 0a 44 61826b39dd 1:0"~Da
0120 74 65 3a 20 54 68 75 2c 20 31 37 20 53 65 70 20te: Thu, 17 Sep
0130 32 38 32 36 20 31 34 3a 32 35 3a 31 33 20 47 4d2026 14: 25:13 GM
0140 54 0d 0a 43 6f 6e 74 65 6e 74 2d 52 61 6e 67 65T~Conte nt-Rang
0150 3a 20 62 79 74 65 73 20 30 2d 36 38 33 37 34 33: bytes 0-683743
0160 2f 36 38 33 37 34 34 0d 0a 43 6f 6e 74 65 6e 74/683744~Conte
0170 2d 4c 65 6e 67 74 68 3a 20 36 38 33 37 34 34 0d-Length: 683744

Step 4: Look at the TCP termination process (FIN, ACK packets).

The connection was closed using FIN, ACK packets.

stream eq 53

Time	Source	Destination	Protocol	Length	Info
542	47.849562700	34.223.124.45	HTTP	161	HTTP/1.1 200 OK (text/html)
548	47.891330200	192.168.100.125	HTTP	484	GET /favicon.ico HTTP/1.1
549	48.122389000	34.223.124.45	HTTP	479	HTTP/1.1 200 OK (PNG)
511	47.127460300	192.168.100.125	TCP	66	64955 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
512	47.358396600	34.223.124.45	TCP	66	80 → 64955 [SYN, ACK] Seq=0 Ack=1 Win=26883 Len=0 MSS=1412 SACK_PERM WS=128
513	47.358673000	192.168.100.125	TCP	54	64955 → 80 [ACK] Seq=1 Ack=1 Win=65280 Len=0
515	47.594548900	34.223.124.45	TCP	54	80 → 64955 [ACK] Seq=1 Ack=523 Win=28032 Len=0
541	47.849325200	34.223.124.45	TCP	1466	80 → 64955 [ACK] Seq=580 Ack=1046 Win=29056 Len=1412 [TCP PDU reassembled in 542]
543	47.849670600	192.168.100.125	TCP	54	64955 → 80 [ACK] Seq=1046 Ack=2099 Win=65280 Len=0
551	48.176277000	192.168.100.125	TCP	54	64955 → 80 [ACK] Seq=1476 Ack=2515 Win=65024 Len=0
559	53.135668100	34.223.124.45	TCP	54	80 → 64955 [FIN, ACK] Seq=2515 Ack=1476 Win=38208 Len=0
560	53.136248800	192.168.100.125	TCP	54	64955 → 80 [ACK] Seq=1476 Ack=2516 Win=65024 Len=0

Name: 559: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface II, Src: HuaweiTechno_ac:e6:82 (10:c1:72:ac:e6:82), Dst: Intel_f0:71:4d (e1:71:4d:00:00:00:00) Internet Protocol Version 4, Src: 34.223.124.45, Dst: 192.168.100.125
Transmission Protocol, Src Port: 80, Dst Port: 64955, Seq: 2515, Ack: 1476, Window: 0
Source Port: 80
Destination Port: 64955
[Stream index: 53]
[Stream Packet Number: 14]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 0]
Sequence Number: 2515 (relative sequence number)
Sequence Number (raw): 2871883000
[Next Sequence Number: 2516 (relative sequence number)]
Acknowledgment Number: 1476 (relative ack number)

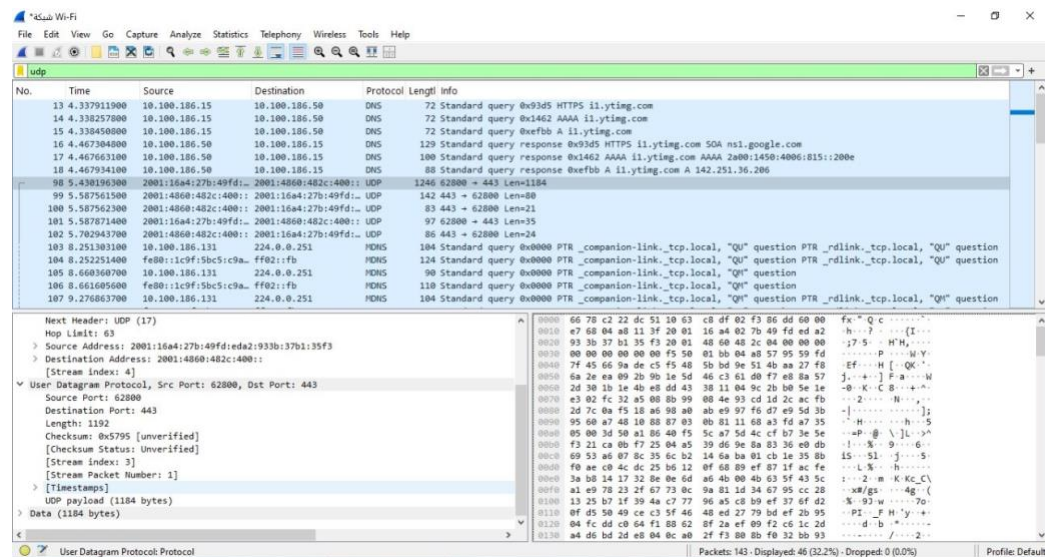
0000 e8 f4 08 f0 71 4d 10 c1 72 ac e6 82 08 00 45 00gM~r~...E~
0010 00 28 57 cb 40 00 f5 06 69 d2 22 df 7c 2d c0 a8[w@~...i~...~
0020 64 7d 00 50 fd 0b ab 2d 74 f8 6d 0f f6 ce 50 11d~P~...t~m~P~
0030 00 ec 68 a5 00 00h~...

Part 3: Capturing and Analyzing UDP Traffic

Task 1: Generate UDP traffic and capture packets

UDP traffic was generated by streaming a video on YouTube while Wireshark was capturing packets on the active network interface.

Task 2: Filter and analysis UDP Packets



Step 4: Observe the source and destination ports, length, and data.

A UDP packet was selected and analyzed. The following information was observed:

- Source Port: 62800
- Destination Port: 443
- Length: 1192
- Data: 1184 bytes of payload data, associated with HTTPS/QUIC traffic over UDP."

Step 5: Compare UDP and TCP Headers

UDP has a simpler header than TCP.

The UDP header contains only four main fields: Source Port, Destination Port, Length, Checksum

TCP has a larger and more complex header because it contains additional fields for connection management, sequencing, acknowledgments, flags, and flow control.

Part 4: Comparing TCP and UDP by filling in the following tables.

Task 1: Fill in the following table and provide reasons.

	TCP or UDP	Reasons
Reliability and Connection Establishment	TCP	TCP establishes a connection and provides reliable delivery using acknowledgments and retransmission.
Data Integrity and Ordering	TCP	TCP ensures reliable delivery and delivers data in the correct order.

Task 2: Identify the use Cases and Performance of TCP and UDP.

	TCP	UDP
Use cases	• HTTP/HTTPS, web browsing, email, file transfer • Applications that need reliable delivery	• DNS, streaming, VoIP, online gaming • Applications that prioritize speed over reliability
Performance	• Slower than UDP due to connection setup (Three-Way Handshake), acknowledgments, retransmissions, and flow control. • Reliable but higher overhead.	• Faster than TCP because it is connectionless and has minimal header overhead. • Unreliable, packets may be lost or arrive out of order.